



MÓDULO 13: Introducción al Desarrollo de Software con Agentes en Antigravity-IDE

Curso: Python para Ciencia Abierta (CSIC)

Autor: Gustavo Liñán Cembrano

Fechas: 5 a 8 de Octubre de 2026

Tiempo Estimado de Impartición: 1 hora

Módulo: 13 - Paradigma de Desarrollo con Agentes IA, SDK de Gemini (`google-generativeai`), Bucle Percepción-Acción-Verificación (PAV), Reglas (`AGENTS.md`), Habilidades (`Skills`) y Function Calling con Gemini

1. Objetivos de Aprendizaje

- **Evolución del Desarrollo Asistido por IA:** Comprender la transición desde la asistencia pasiva (*Ghost Text*) y la refactorización hacia la **Programación Basada en Agentes Autónomos** (*Agentic Coding*).
- **El Bucle Percepción - Acción - Verificación (PAV):** Analizar la arquitectura formal de agentes de código bajo el patrón ReAct (*Reasoning & Acting*).
- **Construcción Paso a Paso de un Agente con Gemini LLM:**
 - **Paso 1:** Definición de Reglas y Directivas de Comportamiento en la **raíz del proyecto** (`.agents/rules/`).
 - **Paso 2:** Creación de Habilidades Contextuales (*Skills* en `.agents/skills/`).
 - **Paso 3:** Definición de Herramientas ejecutables (*Tools*) conectadas con el **servidor OAI-PMH REAL de Digital.CSIC** (visto en el Módulo 10).
 - **Paso 4:** Implementación de la clase `OpenScienceAgent` conectada con el **SDK oficial de Gemini (`google.generativeai`)** para ejecutar **Function Calling (Tool Calling)** en tiempo real.
 - **Paso 5:** Ejecución interactiva en vivo con publicaciones reales del CSIC y auditoría de trazabilidad.

2. De la Asistencia Pasiva a la Programación Basada en Agentes

A lo largo del curso hemos utilizado el autocompletado inteligente (*Ghost Text*) y la refactorización contextual (`Cmd+K`). Sin embargo, el desarrollo asistido por **Agentes Autónomos en Antigravity-IDE** permite delegar tareas complejas y multi-paso al modelo de lenguaje Gemini.

Comparativa de Paradigmas en Antigravity-IDE

Característica	Autocompletado Inline (Tab)	Refactorización (Cmd+K)	Agente Autónomo (Antigravity Agent)
Nivel de Autonomía	Pasivo (Reactivo a la escritura)	Semi-autónomo (A nivel de bloque/archivo)	Alto (Autónomo en todo el proyecto)
Contexto de Análisis	Línea actual / Archivo abierto	Archivo actual / Selección	Repositorio completo, logs, entorno virtual y herramientas
Capacidad de Ejecución	Ninguna	Ninguna	Ejecuta comandos shell, tests, herramientas y lee tracebacks
Flujo de Trabajo	Sugerencia de código	Edición directa de código	Bucle iterativo Percepción-Acción-Verificación (PAV)

El Bucle Formal Percepción - Acción - Verificación (PAV)

Un agente inteligente no adivina respuestas; opera iterativamente sobre su entorno (el proyecto Python y las APIs de investigación) siguiendo un bucle de 3 etapas:

1. **Percepción ($\mathcal{P}$)**: El agente analiza la petición del usuario y examina el estado del entorno (archivos, respuestas de red, errores).
2. **Acción ($\mathcal{A}$)**: El agente selecciona y ejecuta una **Herramienta** (*Tool Call*) específica o edita el código.
3. **Verificación ($\mathcal{V}$)**: El agente comprueba los resultados mediante aserciones (`assert`), validez de formatos y control defensivo de errores, corrigiendo su curso si falla algo.

$$\begin{aligned} &\text{\text{Entorno}} \ \mathcal{E}_t \rightarrow \text{\text{Observar}} \ \text{\text{Percepción}} \ \mathcal{P}_t \rightarrow \text{\text{Razonamiento Gemini}} \ \text{\text{Acción}} \ \mathcal{A}_t \\ &\rightarrow \text{\text{Ejecución Herramienta}} \ \text{\text{Verificación}} \ \mathcal{V}_t \\ &\rightarrow \text{\text{Feedback}} \ \mathcal{E}_{t+1} \end{aligned}$$

3. Construcción del Agente Paso a Paso

PASO 1: Configurar las Directivas y Reglas del Agente en la Raíz del Proyecto (`.agents/`)

Los agentes leen sus reglas de comportamiento desde el directorio `.agents/` ubicado en la raíz del repositorio.

Podríamos crear el archivo manualmente o simplemente ejecutar la celda de abajo que escribe el archivo de reglas de comportamiento que va a tener el agente.

Determinaremos la raíz del espacio de trabajo y crearemos el archivo

`.agents/rules/curacion_abierta.md`:

```
In [ ]: import os

# Determinar dinámicamente la raíz del espacio de trabajo
current_dir = os.getcwd()
project_root = os.path.abspath(os.path.join(current_dir, "..")) if os.path.b

# Definir la ruta exacta en la raíz del repositorio: .agents/rules/
rules_dir = os.path.join(project_root, ".agents", "rules")
os.makedirs(rules_dir, exist_ok=True)

#PRUEBA A PONERLE ALGUNA REGLA TUYA!!!

rule_content = """# Regla de Dominio: Curación de Publicaciones de Ciencia A

## Directivas Principales:
1. **Honestidad en Metadatos:** NUNCA inventar títulos, autores ni DOIs. Toc
2. **Verificación de Identificadores:** Validar siempre el formato de DOIs (
3. **Licencias Abiertas:** Promover el uso de licencias Creative Commons (CC
"""

rule_path = os.path.join(rules_dir, "curacion_abierta.md")
with open(rule_path, "w", encoding="utf-8") as f:
    f.write(rule_content)

print(f"✅ PASO 1 Completado: Regla creada en la RAÍZ del proyecto: '{rule_p
```

PASO 2: Definir una Habilidad Contextual en la Raíz (`.agents/skills/`)

Las **Habilidades (Skills)** son paquetes reutilizables situados en

`.agents/skills/<nombre_skill>/SKILL.md`.

Contienen la cabecera **YAML** (`name`, `description`) y el protocolo de actuación:

```
In [ ]: # Definir la ruta en la raíz del proyecto: .agents/skills/open-science-curat
skill_dir = os.path.join(project_root, ".agents", "skills", "open-science-cu
os.makedirs(skill_dir, exist_ok=True)

skill_content = """____
name: open-science-curator
description: Instrucciones especializadas para consultar el servidor OAI-PMH
____

# Skill: Curador de Ciencia Abierta (CSIC)
```

```

## Protocolo de Actuación:
1. Recibir identificador persistente (DOI o Handle de Digital.CSIC).
2. Validar sintaxis con expresiones regulares (`re`).
3. Realizar petición HTTP al servidor OAI-PMH de Digital.CSIC (`https://digi
4. Extraer elementos Dublin Core (`dc:title`, `dc:creator`, `dc:date`, `dc:r
5. Auditar concordancia con la política de Acceso Abierto del CSIC.
6. Formatear la cita resultante en BibTeX para manuscritos en LaTeX.
"""

skill_path = os.path.join(skill_dir, "SKILL.md")
with open(skill_path, "w", encoding="utf-8") as f:
    f.write(skill_content)

print(f"✅ PASO 2 Completado: Habilidad de curación creada en la RAÍZ del pr

```

📌 PASO 3: Herramientas Ejecutables (Tools) y Módulos de Python (lib/agent_tools.py)

💡 NOTA ARQUITECTÓNICA SOBRE AGENTES E INTELIGENCIA ARTIFICIAL:

En un sistema de agentes profesional como **Antigravity-IDE**, la arquitectura consta de 3 capas claramente diferenciadas:

1. **Directivas y Reglas (.agents/rules/*.md)**: Establecen restricciones éticas y de dominio en lenguaje natural (creadas en el **Paso 1**).
2. **Habilidades Contextuales (.agents/skills/<skill>/SKILL.md)**: Definen el protocolo de actuación en lenguaje natural y metadatos YAML (creadas en el **Paso 2**).
3. **Herramientas Ejecutables (Tools / Módulos de Python / Servidores MCP)**: Código ejecutable real (`requests` , `bs4` , `pandas`) que realiza las operaciones (consultar servidores OAI-PMH, parsear XML, formatear BibTeX).

Formalización de Producción: Las herramientas ejecutables se formalizan en la librería reutilizable del proyecto (`lib/agent_tools.py`) y en el directorio de la habilidad (`.agents/skills/open-science-curator/scripts/agent_tools.py`), importándolas para que el modelo **Gemini** las invoque mediante **Function Calling**.

```

In [ ]: import os
import time
import pandas as pd
from lib.mylib import linea
from lib.agent_tools import (
    tool_validate_identifier,

```

```

    tool_fetch_digital_csic_real,
    tool_audit_open_access,
    tool_format_bibtex,
    tool_generate_summary_report
)

current_dir = os.getcwd()
csv_path = os.path.join(current_dir, '..', 'DATASETS', 'asistentes_handles.csv')
handles = pd.read_csv(csv_path)['handle'].astype(str).str.strip().tolist()

results = []
linea()
print("🚀 INICIANDO AUDITORÍA Y CURACIÓN BATCH CON EL AGENTE para digital.CS")
linea()

for handle in handles[:3]:
    print("")
    print(f"🔍 PROCESANDO: {handle}...")

    val = tool_validate_identifier(handle)
    if not val['valid']:
        print(f"❌ Identificador inválido: {handle}")
        continue

    data = tool_fetch_digital_csic_real(handle)
    if not data['found']:
        print(f"⚠️ No se encontraron metadatos para {handle}")
        continue

    audit = tool_audit_open_access(data['rights'], data.get('rights_license'))
    data['audit'] = audit

    bibtex = tool_format_bibtex(data['title'], data['authors'], data['year'])
    data['bibtex'] = bibtex

    results.append(data)

    print(f"    ✓ Título: {data['title'][:60]}...")
    print(f"    ✓ Autores: {'', ' '.join(data['authors'][:2])}")
    print(f"    ✓ dc.rights: {audit['dc_rights']}")
    print(f"    ✓ dc.rights.license: {audit['dc_rights_license']}")
    print(f"    ✓ Estado Acceso Abierto: {audit['compliance_status']}")
    time.sleep(0.3)

linea()
print(f"🎉 Demostración de herramientas completada: {len(results)} registros")

```

💡 NOTA DIDÁCTICA: ¿POR QUÉ EXISTEN DOS UBICACIONES PARA `agent_tools.py`?

En este desarrollo observarás que los scripts ejecutables de las herramientas se guardan en dos ubicaciones dentro del repositorio:

1. **lib/agent_tools.py (Librería Estándar del Proyecto):**
Permite una importación limpia y directa dentro del entorno Python del cuaderno mediante `from lib.agent_tools import ...`, aprovechando el paquete `lib` del proyecto sin necesidad de manipular rutas relativas complejas.
2. **.agents/skills/open-science-curator/scripts/agent_tools.py (Empaquetado Autónomo de la Skill):**
Cumple con el principio de **Portabilidad de Habilidades (Self-Contained Skills)** de Antigravity. Al incluir la lógica ejecutable dentro de la propia subcarpeta de la habilidad junto a su `SKILL.md`, la habilidad se vuelve **100% autónoma**. Esto permite copiar la carpeta `.agents/skills/open-science-curator/` a cualquier otro repositorio o centro del CSIC sin perder ninguna funcionalidad.

PASO 4: Implementación de la Clase `OpenScienceAgent` (Orquestador con SDK Gemini)

ACLARACIÓN DIDÁCTICA: ¿POR QUÉ EXISTE LA CLASE `OpenScienceAgent` JUNTO A LAS HERRAMIENTAS `Tools` ?

1. **Las Herramientas (`lib/agent_tools.py`):** Son funciones de Python puras, deterministas e independientes (`tool_fetch_digital_csic_real`, `tool_audit_open_access`, `tool_generate_summary_report`).
2. **La Clase Agente (`OpenScienceAgent`):** Representa la entidad inteligente (el Agente autónomo). Su función es:
 - Cargar el contexto del proyecto (Reglas `.agents/rules/` y Habilidades `.agents/skills/`).
 - Conectarse con el SDK de Gemini (`google.genai`).
 - Ofrecer un método principal `process_publication_handle()` que envía el prompt y resuelve dinámicamente las herramientas mediante **Function Calling**.
 - Ofrecer el método `generate_summary_report()`, el cual formatea la respuesta del agente e invoca a `tool_generate_summary_report()` cuando recibe un conjunto de publicaciones.

```
In [ ]: import json
        from dotenv import load_dotenv
```

```

from google import genai
from google.genai import types

# 1. Cargar el archivo .env-->Muy importante la gestion de SECRETOS!!!!
env_loaded = load_dotenv()
# 2. Verificar qué clave API específica se ha cargado
key_name = "GEMINI_API_KEY"
api_key = os.environ.get(key_name)
if api_key:
    # Enmascarar la clave para no exponerla completa en la pantalla (ej: AIza...
    masked_key = f"{api_key[:2]}...{api_key[-3:]}"
    print(f"✅ Variable de entorno '{key_name}' cargada con éxito desde .env")
else:
    print(f"⚠️ No se encontró la variable '{key_name}' en el archivo .env")

```

load_dotenv() es una función de la librería python-dotenv (incluida en la línea 32 de nuestro requirements.txt).

Su objetivo fundamental es cargar de forma segura las variables de entorno confidenciales (como claves de API, contraseñas o tokens de acceso a bases de datos) desde un archivo local oculto llamado .env directamente hacia la memoria del sistema en Python (os.environ).

```

In [ ]: from typing import Dict, Any, List, Optional
import os
from google import genai
from google.genai import types
from lib.agent_tools import (
    tool_validate_identifier,
    tool_fetch_digital_csic_real,
    tool_audit_open_access,
    tool_format_bibtex,
    tool_generate_summary_report
)

class OpenScienceAgent:
    """
    Agente Autónomo de Curación para Ciencia Abierta en el CSIC.
    Orquesta el LLM Gemini (SDK `google-genai`), reglas `.agents/rules/`,
    habilidades `.agents/skills/` y herramientas ejecutables (`lib/agent_tools`)
    (c) Gustavo Liñán Cembrano
    """
    def __init__(self, agent_name: str = "CSIC-Gemini-Agent", model_name: str = "gemini-1.5-flash"):
        self.agent_name = agent_name
        self.model_name = model_name
        self.history = []

        # 1. Cargar explícitamente las Reglas desde .agents/rules/
        self.rules_path = os.path.join(project_root, ".agents", "rules", "rules.json")
        self.rules_text = self._load_file_content(self.rules_path, fallback="")

        # 2. Cargar explícitamente la Habilidad desde .agents/skills/
        self.skill_path = os.path.join(project_root, ".agents", "skills", "skills.json")
        self.skill_text = self._load_file_content(self.skill_path, fallback="")

```

```

# Combinar Directivas y Habilidad como System Instruction para Gemini
self.system_instruction = f"{self.rules_text}\n\n{self.skill_text}"

# 3. Registrar Herramientas ejecutables para el LLM
self.tools = [
    tool_validate_identifier,
    tool_fetch_digital_csic_real,
    tool_audit_open_access,
    tool_format_bibtex
]

# 4. Inicializar cliente Gemini si existe la API Key
self.api_key = os.environ.get("GEMINI_API_KEY")
if self.api_key:
    self.client = genai.Client(api_key=self.api_key)
    self._log(f"Agente '{self.agent_name}' listo con llamadas REALES")
else:
    self.client = None
    self._log(f"⚠️ GEMINI_API_KEY no configurada. Habilitado modo re

self._log(f"📖 Reglas configuradas desde: '{self.rules_path}' ({len(
self._log(f"💡 Habilidad configurada desde: '{self.skill_path}' ({le

def _load_file_content(self, filepath: str, fallback: str) -> str:
    """Lee el contenido de un archivo en el espacio de trabajo."""
    if os.path.exists(filepath):
        with open(filepath, "r", encoding="utf-8") as f:
            return f.read()
    return fallback

def _log(self, message: str) -> None:
    """Mantiene la trazabilidad e historial del agente."""
    self.history.append(message)
    print(f"🗣️ [{self.agent_name}] {message}")

def process_publication_handle(self, handle_input: str) -> Dict[str, Any]:
    """
    Ejecuta la orquestación del agente para un Handle dado.
    """
    self._log(f"=== INICIANDO TAREA: Curación Abierta para Handle '{handle_input}'")

    prompt = f"""Por favor, cura la publicación de Digital.CSIC con Handle '{handle_input}'.
    Sigue las reglas del proyecto y el protocolo de la HERRAMIENTA.
    1. Valida el formato del identificador '{handle_input}'.
    2. Obtén los metadatos reales del servidor OAI-PMH con el identificador '{handle_input}'.
    3. Audita la licencia de acceso abierto.
    4. Genera la cita formateada en BibTeX para LaTeX."""

    if self.client:
        self._log(f"🧠 Enviando Prompt + System Instructions + 4 Tools a Gemini")
        config = types.GenerateContentConfig(
            system_instruction=self.system_instruction,
            tools=self.tools,
            temperature=0.1
        )

```

```

try:
    response = self.client.models.generate_content(
        model=self.model_name,
        contents=prompt,
        config=config
    )
    self._log("✅ Respuesta recibida de Gemini con resolución au
    return {
        "status": "SUCCESS",
        "mode": "REAL_GEMINI_LLM",
        "model": self.model_name,
        "handle": handle_input,
        "llm_response": response.text,
        "applied_rule": "curacion_abierta.md",
        "applied_skill": "open-science-curator"
    }
except Exception as e:
    self._log(f"⚠️ Error en la llamada a la API de Gemini: {e}."

# Modo respaldo si no hay API Key o falla la llamada a Gemini
val_res = tool_validate_identifier(handle_input)
if not val_res.get("valid"):
    return {"status": "ERROR", "reason": val_res.get("error")}
fetch_res = tool_fetch_digital_csic_real(val_res.get("handle"), handl
if not fetch_res.get("found"):
    return {"status": "ERROR", "reason": fetch_res.get("error")}

audit = tool_audit_open_access(fetch_res["rights"], fetch_res.get("r
fetch_res["audit"] = audit
fetch_res["bibtex"] = tool_format_bibtex(fetch_res["title"], fetch_r

return fetch_res

def generate_summary_report(self, result_or_results: Any) -> Dict[str, A
"""
Muestra la respuesta del agente o delega en `tool_generate_summary_r
"""
if isinstance(result_or_results, list):
    return tool_generate_summary_report(result_or_results, author_na
elif isinstance(result_or_results, dict):
    if result_or_results.get("status") == "SUCCESS" and "llm_respons
        print(f"\n--- RESPUESTA AUTÓNOMA DEL AGENTE {self.agent_name
        print(result_or_results.get("llm_response"))
        return result_or_results
    elif "handle" in result_or_results and "found" in result_or_resu
        return tool_generate_summary_report([result_or_results], aut
    else:
        reason = result_or_results.get("reason", result_or_results.g
        print(f"\n⚠️ INFORME DEL AGENTE: {reason}")
        return result_or_results
else:
    print(f"\n📄 Resultado del agente: {result_or_results}")
    return {"status": "info", "result": str(result_or_results)}

```

```
In [ ]: agent = OpenScienceAgent(agent_name="CSIC-Gemini-Agent")
```

PASO 5: Pruebas de Ejecución en Vivo con Publicaciones Reales de Digital.CSIC

Ejecutaremos el agente con los Handles reales e imprimiremos el informe con `agent.generate_summary_report(result)`. Vamos a leer algunos handles de digital.CSIC de los que sois autores o pertenecen a vuestro centro si no he encontrado publicaciones vuestras.

```
In [ ]: import os
import pandas as pd

# Cargar dinámicamente el archivo CSV de Handles de los asistentes y limpiar
current_dir = os.getcwd()
csv_path = os.path.join(current_dir, "..", "DATASETS", "asistentes_handles.csv")

df_handles = pd.read_csv(csv_path)
handles = df_handles['handle'].astype(str).str.strip().tolist()

print(f"✅ Se han cargado {len(handles)} identificadores Handle limpios desc")
for handle in handles:
    print(handle)
```

```
In [ ]: import os
import time
import pandas as pd
from lib.mylib import linea
from lib.agent_tools import (
    tool_validate_identifier,
    tool_fetch_digital_csic_real,
    tool_audit_open_access,
    tool_format_bibtex,
    tool_generate_summary_report
)

current_dir = os.getcwd()
csv_path = os.path.join(current_dir, '..', 'DATASETS', 'asistentes_handles.csv')
handles = pd.read_csv(csv_path)['handle'].astype(str).str.strip().tolist()

results = []
linea()
print("🚀 INICIANDO AUDITORÍA Y CURACIÓN BATCH CON EL AGENTE para digital.CSIC")
linea()

for handle in handles:
    print("")
    print(f"🔍 PROCESANDO: {handle}...")

    val = tool_validate_identifier(handle)
    if not val['valid']:
        print(f"❌ Identificador inválido: {handle}")
        continue

    data = tool_fetch_digital_csic_real(handle)
```

```

if not data['found']:
    print(f"⚠️ No se encontraron metadatos para {handle}")
    continue

audit = tool_audit_open_access(data['rights'], data.get('rights_license'))
data['audit'] = audit

bibtex = tool_format_bibtex(data['title'], data['authors'], data['year'])
data['bibtex'] = bibtex

results.append(data)

print(f"    ✓ Título: {data['title'][:60]}...")
print(f"    ✓ Autores: {'', '.join(data['authors'][:2])}")
print(f"    ✓ dc.rights: {audit['dc_rights']}")
print(f"    ✓ dc.rights.license: {audit['dc_rights_license']}")
print(f"    ✓ Estado Acceso Abierto: {audit['compliance_status']}")
time.sleep(0.3)

linea()
print(f"🎉 Demostración de herramientas completada: {len(results)} registros")

```



Verificación Defensiva ante Identificadores Inválidos

Probemos cómo actúa el agente ante un identificador erróneo o inexistente.

```

In [ ]: # Prueba defensiva con un identificador erróneo
print("=====")
result_error = agent.process_publication_handle("HANDLE_INEXISTENTE_999")
agent.generate_summary_report(result_error)

```

4. El Estándar Emergente: Model Context Protocol (MCP) en Antigravity-IDE

En los pasos anteriores hemos creado herramientas (*Tools*) escribiendo funciones Python locales y registrándolas con `google.genai (tools=[...])`. Sin embargo, en el ecosistema **Antigravity-IDE** y la IA de última (o casi) generación, el estándar universal para conectar agentes con datos se denomina **MCP (Model Context Protocol)**.



¿Qué diferencia hay entre Function Calling y MCP?

- **Function Calling (Nivel de Código):** Defines una función Python local (`def tool_fetch_digital_csic_real()`) en tu cuaderno o script y se la pasas explícitamente al cliente LLM en Python.
- **MCP (Nivel de Infraestructura del IDE):** Es un protocolo cliente-servidor abierto (basado en `stdio` o HTTP/JSON-RPC). Permite que herramientas nativas (sistema de archivos, bases de datos, APIs de Git o servidores OAI-PMH) se conecten

directamente a **Antigravity-IDE** de forma transparente, sin necesidad de escribir código *glue* en Python.

Paso 1: Configurar Servidores MCP en el Proyecto (`.agents/mcp_config.json`)

Para declarar qué servidores MCP debe cargar **Antigravity-IDE** automáticamente al abrir tu repositorio, se crea el archivo `.agents/mcp_config.json` en la raíz:

```
{
  "mcpServers": {
    "filesystem-csic": {
      "command": "npx",
      "args": [
        "-y",
        "@modelcontextprotocol/server-filesystem",
        "./DATASETS"
      ]
    }
  }
}
```

Nota: Antigravity-IDE incluye además **herramientas MCP nativas integradas** (acceso a archivos, terminal y contexto del proyecto) activas por defecto en el panel de chat desde el primer instante.

Paso 2: Interacción Directa en Lenguaje Natural (Chat de Antigravity-IDE)

Como investigador o desarrollador, **no necesitas escribir código** `requests` u `open()`. Simplemente abres el panel de chat (`Cmd + L`) y le pides al Agente:

 **Prompt en el Chat de Antigravity-IDE:**

"Agente, inspecciona la carpeta ``DATASETS/``, lee el archivo ``asistentes_handles.csv`` y muéstrame los Handles de Digital.CSIC registrados."

Otro ejemplo: Agente, usando las funciones disponibles y las que necesites crear y el endpoint OAI-PMH de `lib/digital_csic.py`, dame los 5 últimos registros subidos al repo Digital.CSIC en formato APA

---> Lo hace mal, le damos pista

Agente lo has hecho mal. Para encontrar los 5 ultimos registros, vete a la ultima pagina disponible del resumptionToken y recupera los 5 handles mayores.

---> Se lo complicamos

Agente, serias capaz de hacer lo mismo sin usar el conopcimiento que tienes en la librería digital_csic.py? Solo te permito emplear que sabes cual es el endpoint de digital.csic

Paso 3: Trazabilidad Interna del Agente usando MCP

El Agente resuelve la petición ejecutando autónomamente el bucle **Percepción - Acción - Verificación (PAV)** a través del servidor MCP:

1. **Percepción ($\mathcal{P}$):** El Agente analiza tu petición y consulta los servidores MCP activos.
2. **Acción 1 (Herramienta MCP):** Invoca automáticamente la herramienta `list_directory` del servidor MCP para listar `DATASETS/FROM_DIGITALCSIC/`.
3. **Acción 2 (Herramienta MCP):** Invoca `read_file` del servidor MCP para leer el contenido de `input_handles.csv`.
4. **Verificación ($\mathcal{V}$) y Respuesta:** Sintetiza los datos en una tabla ordenada y presenta el resultado en el chat.

Resumen: ¿Por qué MCP es el Futuro?

Con **MCP**, el Agente puede consultar directamente tus archivos, bases de datos SQLite o APIs repositoriales sin que tengas que programar scripts de lectura ni manejar excepciones HTTP a mano.

5. Resumen del Módulo y Sigüientes Pasos

En este módulo hemos abordado de forma práctica y estructurada el **desarrollo asistido por agentes con Gemini LLM en Antigravity-IDE**:

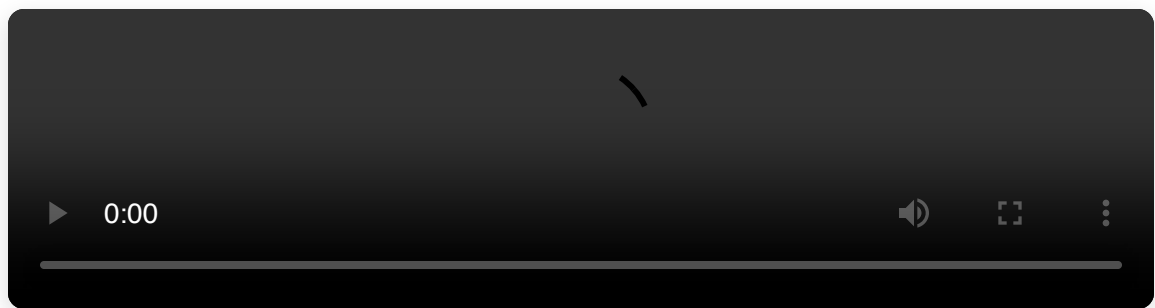
1. **Del Autocompletado Pasivo a los Agentes:** Comprendimos las diferencias entre *Ghost Text* `Tab`, refactorización `Cmd+K` y orquestación autónoma mediante el ciclo **Percepción - Acción - Verificación (PAV)**.
2. **Paso 1 (Reglas en `.agents/rules/`):** Definimos el marco ético y técnico del agente en la raíz del proyecto (`.agents/rules/curacion_abierta.md`).
3. **Paso 2 (Habilidades en `.agents/skills/`):** Empaquetamos conocimiento experto en `.agents/skills/open-science-curator/SKILL.md`.

4. **Paso 3 (Herramientas `Tools` Reales):** Implementamos funciones Python que realizan **consultas HTTP REALES al servidor OAI-PMH de Digital.CSIC** (reutilizando los conocimientos del Módulo 10).
 5. **Paso 4 & 5 (Agente Gemini con `google.genai`):** La clase `OpenScienceAgent` configura Gemini enviando el System Instruction y la lista de Tools para ejecutar **Function Calling / Tool Calling** en tiempo real.
-

Siguiente Módulo: Proyecto Final de GitHub

En el próximo y último módulo del curso (**Módulo 14: Proyecto Final de GitHub**), integraremos todo el conocimiento del curso: estructuras de datos, modularización, manipulación de metadatos con Pandas, gráficos en Matplotlib, empaquetado de librerías, integración continua (CI/CD) y desarrollo guiado por agentes en GitHub.

Videopodcast Resumen del Módulo (13)



Cuestionario de Auto-evaluación y Participación

¡Pon a prueba los conocimientos adquiridos en este módulo! Completa este breve cuestionario interactivo de 5 minutos en Google Forms:

 [Cuestionario Módulo 13 en Google Forms](#)

Tutor Virtual & Libreta de Estudio (Google NotebookLM)

¿Quieres escuchar el resumen conversacional en audio de este módulo o resolver dudas con el tutor virtual del curso?

 [Acceder al Espacio de Estudio Interactivo en NotebookLM](#)
